

Stack e arquitetura da API - BlueTide

Item do quadro: “Definir stack e arquitetura da API” (Sprint 3, entrega 14/09/2026).

Define a tecnologia e o desenho da API que vai servir ao front end o status verde/amarelo/vermelho calculado a partir do boletim do INEA, da chuva recente e da maré.

Sobre o shape da recomendação usado aqui

A task `inf1039-logica-recomendacao`, que define a lógica de cálculo do status, ainda não tem output no branch principal nem em PR aberto no momento em que este documento foi escrito. Por isso os nomes de campo do payload abaixo (`status`, `componentes.boletim_inea`, `componentes.chuva`, `componentes.mare`, os três valores de status `proprio/improprio/sem_dado`) são um shape assumido, não confirmado com aquela task. Quando a lógica de recomendação for definida, os nomes de campo desta API devem ser revisados para bater com o que ela realmente produz. O shape assumido aqui se apoia no que já está documentado em `Sprint-2/Vitor-Rocha/fontes-de-dados.pdf` e no comparativo de similares em `Sprint-1/Vitor-Rocha/analise-similares.md`, que aponta a idade do dado como um diferencial central do projeto: por isso cada componente carrega seu próprio timestamp de atualização.

Stack escolhida: Python + FastAPI

Escolha: FastAPI rodando sobre Uvicorn, com Pydantic para validar entrada e saída.

Por que essa e não outra:

- O projeto já é Python (o próprio `main.py` da raiz, mesmo sendo um stub descartável, mostra que a equipe já trabalha em Python).
- FastAPI gera documentação interativa (Swagger/OpenAPI) automaticamente a partir do código, o que ajuda tanto o time de front end a integrar quanto o professor a avaliar a API sem precisar ler Postman collections à parte.
- A curva de aprendizado é baixa para um time de alunos: poucos arquivos, sem configuração de servidor de aplicação separada, `uvicorn main:app --reload` já sobe tudo localmente.
- Pydantic força o formato do payload verde/amarelo/vermelho a ficar explícito em código, o que evita a API devolver um JSON solto e inconsistente conforme mais gente mexe nela.

O tradeoff aqui é deliberado: FastAPI é menos “enterprise” que uma stack com camadas de serviço, ORM completo e fila de mensagens. Este é um projeto de curso com poucas semanas de sprint e uma única fonte de estado real, o boletim semanal do INEA; adicionar essa complexidade não traria robustez proporcional ao tamanho do problema, só custo de manutenção para o time. Se o projeto crescer para múltiplas cidades ou usuários simultâneos em escala, essa decisão deve ser revisitada, mas não é o caso hoje.

Alternativa considerada e descartada: Flask puro. Ele é ainda mais simples, mas exige escolher e configurar manualmente uma lib de validação e outra de geração de documentação; o ganho de simplicidade inicial não compensa perder a validação de schema embutida do FastAPI, que é justamente o que evita bugs de contrato entre back e front em um time pequeno e sem QA dedicado.

Onde a API deve morar no repositório

O `main.py` da raiz é um exercício de calculadora, não o começo do app real; ele deve ser ignorado e, quando o time achar oportuno, removido em uma tarefa própria (não faz parte deste escopo apagá-lo).

Recomendação: a API real deve viver em uma pasta dedicada na raiz do repositório, por exemplo `backend/`, e não dentro de `Sprint-3/Vitor-Rocha/` nem de nenhuma outra pasta de `sprint/pessoa`. As pastas `Sprint-N/<Nome-Sobrenome>/` são o histórico de entregas de cada sprint por pessoa, não o lugar onde o código do produto deve acumular ao longo do tempo; se o código da API for commitado ali, ele fica implicitamente “de uma pessoa” e sprints futuras não têm um lugar óbvio para continuar o mesmo código. Por isso este documento fica em `Sprint-3/Vitor-Rocha/stack-arquitetura-api/` (é a entrega desta sprint), mas o esqueleto de código abaixo é só uma prova de conceito da forma da API; a implementação real subsequente deve ser movida para `backend/` na raiz assim que o time abrir a tarefa de implementação.

Entradas: boletim do INEA, chuva e maré

As três fontes têm frequências de atualização muito diferentes, então a API não trata todas do mesmo jeito:

- Boletim do INEA (balneabilidade): publicado semanalmente como PDF em `inea.rj.gov.br`, sem API própria. A API deve rodar um job agendado (ex.: uma vez por dia, já que o boletim é semanal mas o dia exato da publicação varia) que baixa o PDF, faz o parsing e grava o resultado em um cache local, junto com o timestamp da coleta. Fazer esse parsing a cada requisição do front end seria lento e desnecessário, já que o dado só muda uma vez por semana.
- Chuva (Open-Meteo Weather API): API REST gratuita, sem necessidade de chave. A API deve fazer polling em intervalo curto (ex.: a cada hora) e cachear o resultado por praia/coordenada, porque o dado muda com frequência mas não precisa ser em tempo real para a decisão de “ir ou não ir”.
- Maré: a Open-Meteo Marine API cobre ondas e temperatura do mar, mas o próprio levantamento de fontes já registra baixa precisão dela para maré; o iScamar, concorrente adjacente analisado no Sprint 1, resolve isso combinando estação harmônica com um serviço de terceiros. Até essa fonte de maré ser definida com mais detalhe, a API deve tratar o dado de maré como um componente plugável e cacheado no mesmo padrão da chuva (polling horário), para não travar o resto da arquitetura nessa decisão em aberto.

A API não calcula tudo a cada request. Ela mantém um cache alimentado por polling em background, diário para o INEA e horário para chuva e maré. Quando o front end pede a recomendação, ela lê esse cache, aplica a lógica de recomendação e devolve o resultado com o

timestamp de cada componente. Isso atende ao ponto que a análise de similares levantou como diferencial do projeto: nenhum concorrente avisa quando o dado está velho, e aqui isso fica explícito no payload por construção.

Endpoints

GET /api/v1/praias

Lista as praias monitoradas, para o front end popular um seletor ou mapa.

```
[
  { "id": "copacabana-posto-6", "nome": "Copacabana - Posto 6", "lat":
-22.9711, "lon": -43.1822 },
  { "id": "ipanema-posto-9", "nome": "Ipanema - Posto 9", "lat":
-22.9868, "lon": -43.2013 }
]
```

GET /api/v1/praias/{praia_id}/recomendacao (endpoint principal)

Devolve o status calculado para uma praia específica. É este endpoint que o front end chama para montar a tela principal do app.

Exemplo de request:

GET /api/v1/praias/copacabana-posto-6/recomendacao

Exemplo de resposta:

```
{
  "praia_id": "copacabana-posto-6",
  "praia_nome": "Copacabana - Posto 6",
  "status": "amarelo",
  "motivo": "Chuva forte nas últimas 18h; dentro da janela de 24h de
restrrição do INEA.",
  "atualizado_em": "2026-09-22T09:00:00-03:00",
  "componentes": {
    "boletim_inea": {
      "status": "proprio",
      "coletado_em": "2026-09-18T00:00:00-03:00",
      "idade_horas": 90
    },
    "chuva": {
      "mm_ultimas_24h": 32.5,
      "dentro_janela_restricao": true,
      "fonte": "open-meteo-weather-api",
      "atualizado_em": "2026-09-22T08:00:00-03:00"
    },
    "mare": {
      "fase": "enchente",
      "altura_m": 0.8,
      "atualizado_em": "2026-09-22T08:00:00-03:00"
    }
  }
}
```

```
}  
}
```

status assume verde, amarelo ou vermelho, seguindo a decisão de produto validada na pesquisa (H18 em Sprint-1/Vitor-Rocha/hipoteses.md): uma resposta única e interpretada, não uma lista de dados técnicos soltos. Os nomes dentro de componentes são o shape assumido descrito na seção acima e devem ser conferidos contra a saída real da task de lógica de recomendação assim que ela existir.

GET /api/v1/praias/{praia_id}/recomendacao/health (opcional, se sobrar tempo)

Endpoint auxiliar só para debug/operação: devolve quando cada fonte (INEA, chuva, maré) foi coletada pela última vez, sem calcular status. Útil para o time perceber rapidamente se o job de polling parou de rodar, sem precisar vasculhar logs.

Próximos passos fora deste escopo

- Confirmar o shape de componentes contra a task inf1039-logica-recomendacao assim que ela tiver output.
- Decidir a fonte definitiva de maré (estação harmônica própria vs. serviço de terceiros, como o iScamar faz).
- Mover a implementação real da API para backend/ na raiz do repositório quando a tarefa de implementação for aberta.